MASTER OF BUSINESS ADMINISTRATION

SEMESTER 3

DADS304

VISUALIZATION

Unit 4

Dashboard Design using R-Shiny

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Dashboard design using R-Shiny	-	1	4
3	Creation of R-Shiny dashboard	1 to 20	2, 3, 4	5 - 18
4	Adding features in R-Shiny dashboard	21 to 51	5	19 - 32
5	Summary	-	-	33
6	Glossary	-	-	33
7	Concept Map	-	-	34
8	Study notes and did you Know	-	-	34
9	Case study	53 to 54	-	35 - 36
10	Terminal Questions	-	-	36
11	Self-Assessment answer	-	-	37
12	Terminal Questions Answers	-	-	37 - 41
13	References	-	-	41

1. INTRODUCTION

The R-Shiny framework is a package from R Studio that makes it easy to build interactive web applications straight from R. R-Shiny offers powerful analysis tools and data manipulation or wrangling. It also offers advanced forecasting packages. and statistical modeling. Through the web applications, a user can visualize data like metadata, bibliographic data, etc… and to create efficient data reports. These visualization tools can stimulate other users to create open repositories and connect either regional, national or international repositories networks. It compiles the user code into the HTML, JavaScript and CSS needed to display users' application on the web.

1.1 Objective

After studying, you should be able to:

- *Discuss the dashboard design using R-Shiny.*
- *Discuss the creation of R-Shiny dashboard.*
- *Identify the features in the R-shiny dashboard.*

INSPIRED BY LIFE

2. DASHBOARD DESIGN USING R-SHINY

2.1 Dashboard

Dashboards are tools that offer current information while employing images to explain the narratives underlying the data. They assist decision-makers in understanding the connections in complex, massive data. They display images in a useful arrangement that makes it easier for the organization to understand and appreciate the data.

2.2 Dashboard Design using R-Shiny

Shiny dashboards give users inside the R environment access to a full web application framework. You may quickly turn your R work, analysis, and visualizations, machine learning models, among other things, into web applications that benefit companies. End-users can use it as a complete application without having any prior knowledge of R. Deliver a comprehensive, user-friendly, and interactive product that enhances your business operations.

You may use Shiny as a dashboard development platform to access a variety of R packages for data research, including the Tidyverse. For the visualization of data and models, you can access advanced graphical features. Add responsiveness and engagement by embedding these images in Shiny dashboards. This can be done by using an interface that R has allowed to communicate with JavaScript-based charting packages.

The administration and structure of code is made easier by taking a look at the Shiny dashboard's design, making use of functions, modules, and packages, and using rapid prototyping. Simple source code controls and smaller, more manageable dashboard components are both possible.

Self-Assessment Questions - 1

1. _____ environment is used in R-Shiny.
2. _____ assist decision-makers in understanding the connections in complex, massive data.

3. CREATION OF R-SHINY DASHBOARD

If you need to summarize or display a lot of information on a single webpage so that everything can be read in one window, a dashboard can be a smart option. Access to a full web application framework is made possible within the R environment via shiny dashboards. A directory containing a R script saved as app. R makes up a Shiny application project. The script is made of code that describes the user interface object and the server function. These two are both supplied as arguments to the shinyApp method. resulting in the construction of either a web app or dashboard as a Shiny app object.

To ensure that your Shiny apps are not only user-friendly but also offer a pleasurable experience for your users, follow the 7 steps listed below.

- Begin with the cause
- Use paper and a pen.
- Use prepared solutions.
- Get motivated
- Prioritize data storytelling
- Test first, then repeat the process.
- Look past UI

Self-Assessment Questions – 2

3. To display a lot of information on a single webpage we use _____
4. The script is made of code that describes _____ and _____

3.1 Begin With The Cause:

Start by considering the following issues while choosing the best UX design for your Shiny app:

- What was the primary impetus behind developing the app?
- Who utilizes the services?

- What are the users going to be able to do with your app? What is the company's vision?
- How have they accomplished it thus far? Are there any tools or processes to which they are already accustomed?

Prior to building the Shiny app, it's critical to know the answers to these questions in order to ensure that the interface will support the main functionality. Knowing the users' identities will enable you to highlight the important information and conceal or even skip the rest. The best way to provide the results—in a table, as a downloadable file, or as a graph—depends entirely on what the users need to complete. Observing how users now complete that task will help you better understand the entire process or possibly identify a potential competitive advantage.

Use paper and a pen:

Nevertheless, it's a wise idea to initially sketch out your ideas before you begin creating your app. The explanation is straightforward: redesigning the wireframes is significantly simpler than altering the UI code.

Although there are many tools available (such as Figma) for creating wireframes and mockups, it is perfectly acceptable to start with simply a pen and paper.The UI components may easily be removed and moved about the page to observe how they work together.

Don't stick with the first design; try different ones, get input from the intended audience, and refine the design.

Use prepared solutions:

There are many tried-and-true packages that can give your Shiny app a polished appearance. You may utilize Appsilon's shiny.fluent to create business applications, especially for environments that lean heavily toward Microsoft.

Get motivated:

Try finding some inspiration if you're having trouble with the page layout. To see how similar capabilities are handled on websites you enjoy, visit them and learn how to navigate easily. You can peruse Shiny demos if you're in the mood for more application cases. Then, you can go to websites that irritate you or on which you simply cannot seem to get the information you seek, and try to comprehend the underlying issue that is affecting user experience.

A good user experience design should adhere to certain principles in UX design if you require a more "formal" strategy.

Prioritise data storytelling:

The majority of Shiny programs include data visualization, making R an excellent choice for data processing and analysis. Although data visualization design is a large subject, there are a few crucial points to take into account to get us started.

4 rapid data display tips:

Type of graph

Use a line graph if the information is primarily concerned with changes over time. Use a bar chart when you want to demonstrate how the levels of various categories differ from one another (and don't forget to always start the bar chart at 0!).

Colors

Choose your colors carefully because an overuse of them might make your graph clumsy to look at and hard to read.

Axes

The axis should be adjusted to correspond to the data; otherwise, it will be challenging to identify the data's variance.

Labels

Consider the labels and consider whether the precise value is crucial. If so, it's a good idea to make the label on the graph clear. However, we can hide the labels and make the graph easier to understand if the relationship between both the series is more significant.

Test first, then repeat the process:

The best approach to determine whether your Shiny app satisfies user expectations is through in-depth user interviews. They are extended user sessions when we ask them to utilise the product to complete a number of tasks. In this manner, we may determine whether there are any persistent issues with the navigation or the usability in general.

The great news is that the app layout can be tested at any point throughout development, even before it begins. One can manually alter the "screens" as the user "performs an action" in the app using wireframes or mockups. Just keep in mind not to put off testing until the last minute. Rebuilding the UI when app development is complete will be expensive.

Look past UI:

It's simple to overlook the fact that the user experience of an app includes more than just how it appears and functions. Check out the following items to get your Shiny app off to a good start:

- Is the design of the app responsive?
- When a user encounters an error message, do they understand what went wrong?
- Is the user notified about the app's status, such as when they are waiting for a calculation and wondering when it will be finished?

Remember that this is simply a starting point for you. In addition to the aesthetic element of the user interface, there are more UX touch-points to take into account the more complicated the app and user flow.

3.2 Creation Of Simple Dashboard:

Installation:

Shiny 0.11 or later is needed for shinydashboard.Run: to install.

```
install.packages("shinydashboard")
```

Fig.1. Installing r-Shiny

Basics:

A dashboard is composed of a body, a sidebar, and a header. Here is the dashboard page's user interface at its most basic.

```
## ui.R ##
library(shinydashboard)

dashboardPage(
  dashboardHeader(),
  dashboardSidebar(),
  dashboardBody()
)
```

Fig.2. Basic syntax for dashboard.

Blank dashboard:

Using the shinyApp() function, you may immediately view it at the R console. (This code can also be used to create a single-file app.)

```
## app.R ##
library(shiny)
library(shinydashboard)

ui <- dashboardPage(
  dashboardHeader(),
  dashboardSidebar(),
  dashboardBody()
)

server <- function(input, output) { }

shinyApp(ui, server)
```

Fig.3.Blank dashboard program.

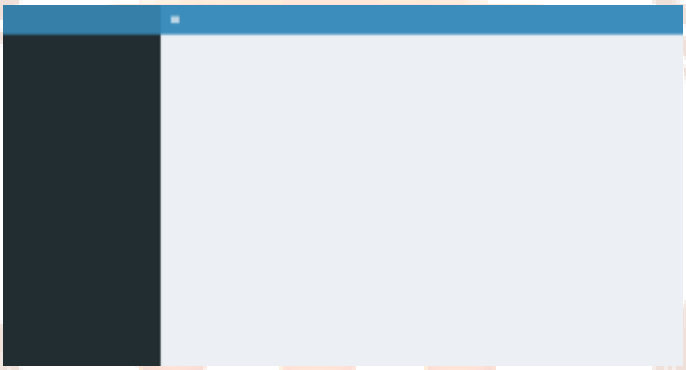

Fig.4.Output of above program.

Self-Assessment Questions – 3

5. A dashboard is composed of _____ , _____ , and _____
6. Blank dashboard can be created by _____ function_

Basic dashboard:

The blank dashboard is obviously not very helpful. We'll need to include elements that are functional. We can include content-filled boxes in the body.

```r
## app.R ##
library(shinydashboard)

ui <- dashboardPage(
  dashboardHeader(title = "Basic dashboard"),
  dashboardSidebar(),
  dashboardBody(
    # Boxes need to be put in a row (or column)
    fluidRow(
      box(plotOutput("plot1", height = 250)),

      box(
        title = "Controls",
        sliderInput("slider", "Number of observations:", 1, 100, 50)
      )
    )
  )
)

server <- function(input, output) {
  set.seed(122)
  histdata <- rnorm(500)

  output$plot1 <- renderPlot({
    data <- histdata[seq_len(input$slider)]
    hist(data)
  })
}

shinyApp(ui, server)
```

Fig.5. Basic Program for dashboard.

Fig.6.Output of the above program.

The sidebar's content can then be added next. We'll add menu options that function like tabs for this example. These work similarly to Shiny's tab Panels in that they display a distinct set of material in the main body when you click on a certain menu item.

There are two tasks that must be completed. You must first add sidebar menu items with the proper **tabNames**.

```
## Sidebar content
  dashboardSidebar(
    sidebarMenu(
      menuItem("Dashboard", tabName = "dashboard", icon = icon("dashboard")),
      menuItem("Widgets", tabName = "widgets", icon = icon("th"))
    )
  )
```

Fig.7. Creating Sidebar.

Add **tabItems** with the corresponding values for **tabName** to the body:

```
## Body content
  dashboardBody(
    tabItems(
      # First tab content
      tabItem(tabName = "dashboard",
        fluidRow(
          box(plotOutput("plot1", height = 250)),

          box(
            title = "Controls",
            sliderInput("slider", "Number of observations:", 1, 100, 50)
          )
        )
      ),

      # Second tab content
      tabItem(tabName = "widgets",
        h2("Widgets tab content")
      )
    )
```

Fig.8. Creating a Body of the dashboard.

The default display, which also appears when the menu item "Dashboard" is clicked:

Fig.9. Default display of dashboard menu.

And this is what appears when you click "Widgets":

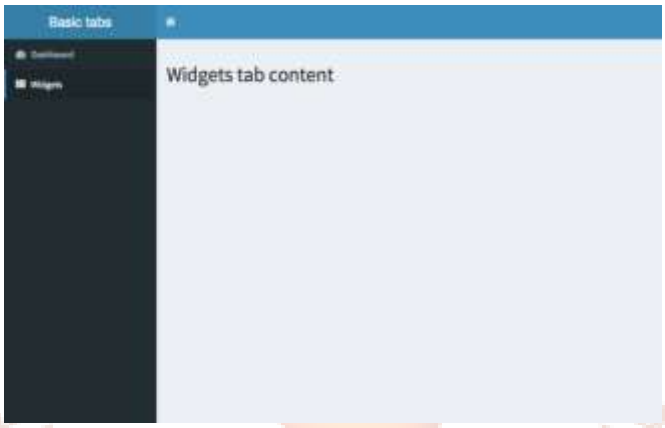

Fig.10. Widgets menu.

3.3 Boxes:

The primary components of dashboard pages are boxes. With the **box()** function, a simple box may be built, and (most) any Shiny UI element can be used as its content.

Also with title and status settings, boxes can also have titles and different colored header bars.

```
box(
  title = "Histogram", status = "primary", solidHeader = TRUE,
  collapsible = TRUE,
  plotOutput("plot3", height = 250)
),

box(
  title = "Inputs", status = "warning", solidHeader = TRUE,
  "Box content here", br(), "More box content",
  sliderInput("slider", "Slider input:", 1, 100, 50),
  textInput("text", "Text input:")
)
```

Fig.11. Program for boxes.

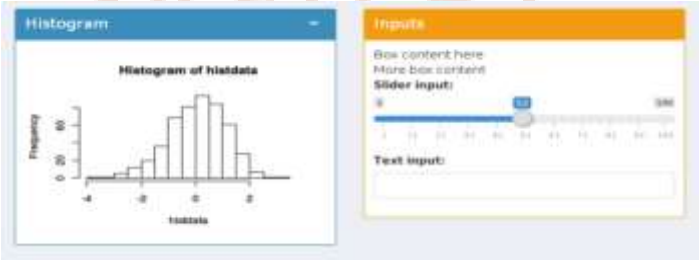

Fig.12. Dashboard Page of boxes.

tabBox:

Use a **tabBox** if you need a container to just have tabs for showing various content sets.

```
body <- dashboardBody(
  fluidRow(
    tabBox(
      title = "First tabBox",
      # The id lets us use input$tabset1 on the server to find the current tab
      id = "tabset1", height = "250px",
      tabPanel("Tab1", "First tab content"),
      tabPanel("Tab2", "Tab content 2")
    ),
    tabBox(
      side = "right", height = "250px",
      selected = "Tab3",
      tabPanel("Tab1", "Tab content 1"),
      tabPanel("Tab2", "Tab content 2"),
      tabPanel("Tab3", "Note that when side=right, the tab order is reversed.")
    )
  ),
  fluidRow(
    tabBox(
      # Title can include an icon
      title = tagList(shiny::icon("gear"), "tabBox status"),
      tabPanel("Tab1",
        "Currently selected tab from first box:",
        verbatimTextOutput("tabset1Selected")
      ),
      tabPanel("Tab2", "Tab content 2")
    )
  )
)
```

Fig.13. Implementation of tabBox code-1

```
shinyApp(
  ui = dashboardPage(
    dashboardHeader(title = "tabBoxes"),
    dashboardSidebar(),
    body
  ),
  server = function(input, output) {
    # The currently selected tab from the first box
    output$tabset1Selected <- renderText({
      input$tabset1
    })
  }
)
```

Fig.14. Implementation of tabBox code-2

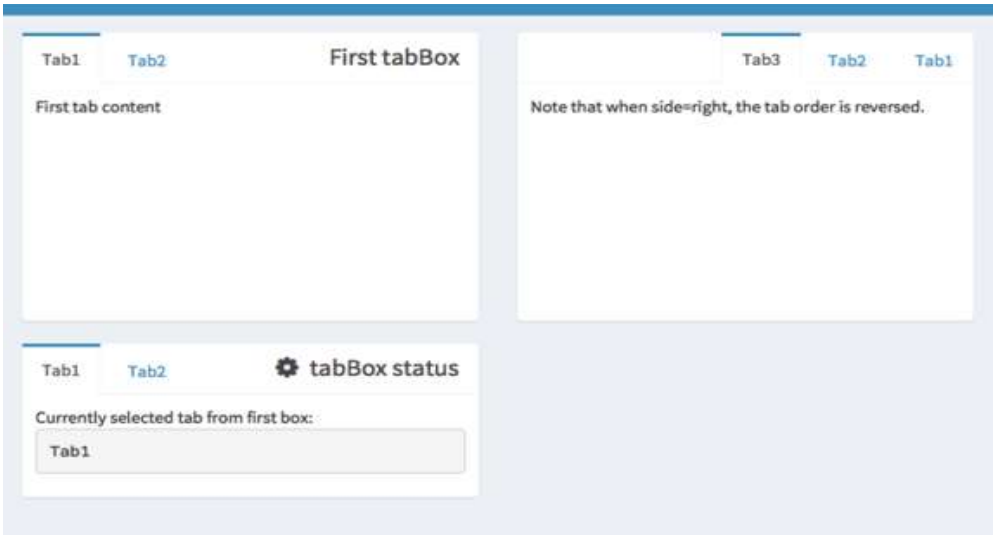

Fig.15. Dashboard of tabBox.

infoBox:

Simple numerical or text values are typically displayed in a particular type of box with an icon.

Fill=FALSE is the default setting for the initial row of infoBoxes, while **fill=TRUE** is used for the second row. Shiny Dashboard includes the auxiliary functions **infoBoxOutput** and **renderInfoBox** for dynamic content because infoBox content is typically dynamic.

```
library(shinydashboard)

ui <- dashboardPage(
  dashboardHeader(title = "Info boxes"),
  dashboardSidebar(),
  dashboardBody(
    # infoBoxes with fill=FALSE
    fluidRow(
      # A static infoBox
      infoBox("New Orders", 10 * 2, icon = icon("credit-card")),
      # Dynamic infoBoxes
      infoBoxOutput("progressBox"),
      infoBoxOutput("approvalBox")
    ),

    # infoBoxes with fill=TRUE
    fluidRow(
      infoBox("New Orders", 10 * 2, icon = icon("credit-card"), fill = TRUE),
      infoBoxOutput("progressBox2"),
      infoBoxOutput("approvalBox2")
    ),

    fluidRow(
      # Clicking this will increment the progress amount
      box(width = 4, actionButton("count", "Increment progress"))
    )
  )
)
```

Fig.16. Implementation of infoBox code-1.

```
server <- function(input, output) {
  output$progressBox <- renderInfoBox({
    infoBox(
      "Progress", paste0(25 + input$count, "%"), icon = icon("list"),
      color = "purple"
    )
  })
  output$approvalBox <- renderInfoBox({
    infoBox(
      "Approval", "80%", icon = icon("thumbs-up", lib = "glyphicon"),
      color = "yellow"
    )
  })

  # Same as above, but with fill=TRUE
  output$progressBox2 <- renderInfoBox({
    infoBox(
      "Progress", paste0(25 + input$count, "%"), icon = icon("list"),
      color = "purple", fill = TRUE
    )
  })
  output$approvalBox2 <- renderInfoBox({
    infoBox(
      "Approval", "80%", icon = icon("thumbs-up", lib = "glyphicon"),
      color = "yellow", fill = TRUE
    )
  })
}

shinyApp(ui, server)
```

Fig.17. Implementation of infoBox code-2.

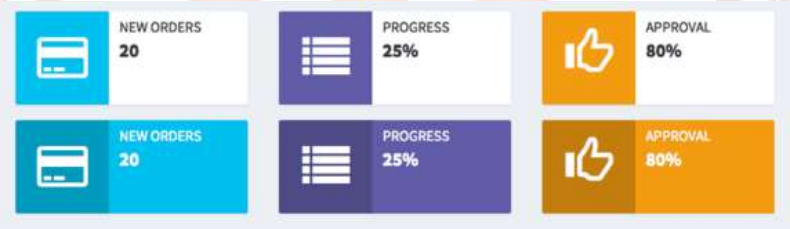

Fig.18. Dashboard of infoBox.

ValueBoxes:

ValueBoxes resemble infoBoxes but are visually distinct from them. The following code will create these valueBoxes. Some of these **valueBoxes** are static and some are dynamic, just like the infoBoxes mentioned before.

```r
library(shinydashboard)

ui <- dashboardPage(
  dashboardHeader(title = "Value boxes"),
  dashboardSidebar(),
  dashboardBody(
    fluidRow(
      # A static valueBox
      valueBox(10 * 2, "New Orders", icon = icon("credit-card")),

      # Dynamic valueBoxes
      valueBoxOutput("progressBox"),

      valueBoxOutput("approvalBox")
    ),
    fluidRow(
      # Clicking this will increment the progress amount
      box(width = 4, actionButton("count", "Increment progress"))
    )
  )
)

server <- function(input, output) {
  output$progressBox <- renderValueBox({
    valueBox(
      paste0(25 + input$count, "%"), "Progress", icon = icon("list"),
      color = "purple"
    )
  })

  output$approvalBox <- renderValueBox({
    valueBox(
      "80%", "Approval", icon = icon("thumbs-up", lib = "glyphicon"),
      color = "yellow"
    )
  })
}

shinyApp(ui, server)
```

Fig.19. Implementation of valueBoxes code.

Fig.20. Dashboard of valueBoxes.

Self-Assessment Questions – 4

7. A simple box can be created using _____ function.
8. Boxes design can be classified as _____ , _____ and _____

INSPIRED BY LIFE

4. ADDING FEATURES IN R-SHINY DASHBOARD

We first need to understand how a Shiny UI is constructed and how it interacts with the HTML of a web page in order to comprehend how the components of a dashboard function together.

Like div() and p(), the HTML tag methods in Shiny return objects that can be rendered as HTML. For instance, when you issue the following instructions at the R terminal, HTML is printed out:

```
# A basic div
div(class = "my-class", "Div content")
```

Div content

```
# Nested HTML tags
div(class = "my-class", p("Paragraph text"))
```

Paragraph text

Fig.21. HTML tag in R-Shiny.

These parts of HTML are used to create the UI for a Shiny app. A collection of utilities created to construct HTML that will generate a dashboard are offered by the shinydashboard package. The dashboard will print out HTML if you copy the UI code for a page in the dashboard (above) and put it into the R console.

4.1. Structure Overview

Three components make up a "shinydashboard": a header, a sidebar, and a body.

Let's look at a straightforward dashboard. You can see that the title is contained in dashboardHeader () and that dashboardSidebar () contains a sidebarMenu(). Output is contained in the dashboardBody ().

The dashboardPage () function's six key components are as follows:

- Skin
- Header
- Sidebar
- Body
- Controlbar
- Footer

Fig.22. Sales Revenue Dashboard Application - UI interface.

We'll now examine each of the six elements that make up a shinydashboard.

- **skin()**

  The color theme is the skin. The backdrop of the sidebar will be light if the skin is light. Depending on the type of app you build, it is simple to select the appearance you like. The plot color should complement the skin you select for your application. The list of skin tones is provided below.

```
skin = c("blue", "blue-light", "black", "black-light", "purple", "purple-light", "green", "green-light", "red", "red-light", "yellow", "yellow-light", "midnight")
```

Fig.23. List of skin tones.

- **header()**

  Dropdown menus and titles are both possible in a header. Here's an illustration:

```
dashboardHeader(title = "My Dashboard")
```

Fig.24. Creating title in header.

The dropdownMenu() function creates the dropdown menus. There are three different types of menus: messages, notifications, and tasks, and each one needs a specific kind of material to be filled with.

Message menus

Values for **from** and **message** are required for a **messageItem** in a message menu. The icon and a notification time string are also under your control. Any text can be the time string.

Fig.25. Message Menu.

Dynamic content

You'll want to make the content dynamic in the majority of circumstances. To put it another way, the HTML content is created on the server and delivered to the client to be rendered.

```
dashboardHeader(dropdownMenuOutput("messageMenu"))
```

Fig.26. Dynamic content.

And on the server side, you would create a **renderMenu** to build the complete menu, as in:

```
output$messageMenu <- renderMenu({
  # Code to generate each of the messageItems here, in a list. This assumes
  # that messageData is a data frame with two columns, 'from' and 'message'.
  msgs <- apply(messageData, 1, function(row) {
    messageItem(from = row[["from"]], message = row[["message"]])
  })

  # This is equivalent to calling:
  #   dropdownMenu(type="messages", msgs[[1]], msgs[[2]], ...)
  dropdownMenu(type = "messages", .list = msgs)
})
```

Fig.27. Implementation of RenderMenu.

Notification menus

Fig.28. Notification menus.

A notification contains a **notificationItem** that contains a text notification. The user can also control the status color and icon.

```
dropdownMenu(type = "notifications",
  notificationItem(
    text = "5 new users today",
    icon("users")
  ),
  notificationItem(
    text = "12 items delivered",
    icon("truck"),
    status = "success"
  ),
  notificationItem(
    text = "Server load at 86%",
    icon = icon("exclamation-triangle"),
    status = "warning"
  )
)
```

Fig.29. Implementation of Notification menus.

Task menus

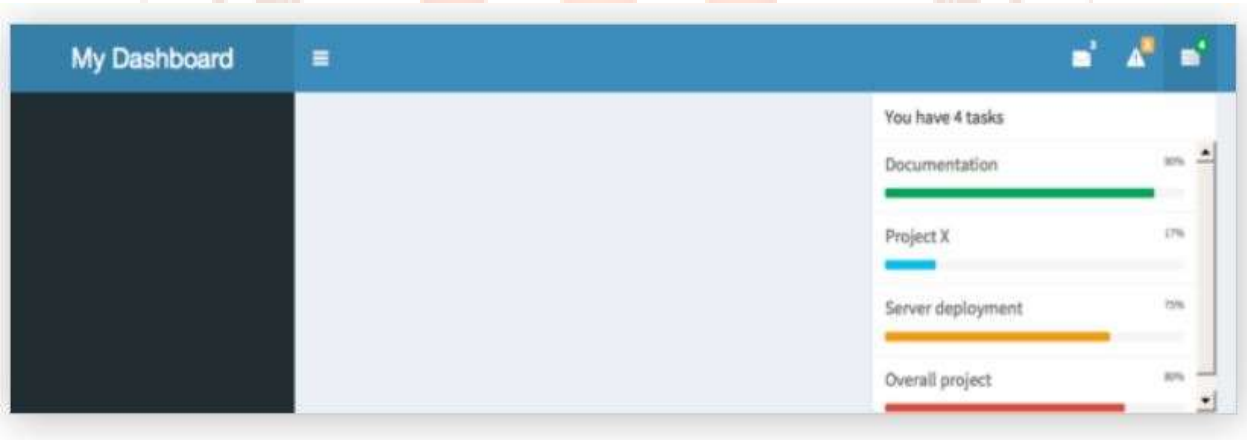

Fig.30. Task menus.

Progress bars and text labels are shown on task items. The bar's color can also be chosen.

```
dropdownMenu(type = "tasks", badgeStatus = "success",
  taskItem(value = 90, color = "green",
    "Documentation"
  ),
  taskItem(value = 17, color = "aqua",
    "Project X"
  ),
  taskItem(value = 75, color = "yellow",
    "Server deployment"
  ),
  taskItem(value = 80, color = "red",
    "Overall project"
  )
)
```

Fig.31. Implementation of Task menus.

Disabling the header

You can prevent a header bar from appearing by using the following command:

```
dashboardHeader(disable = TRUE)
```

Fig.32. Disabling the header.

- **sidebar()**

  Usually, a sidebar is used for rapid navigation. It may also have Shiny inputs like sliders and text inputs, as well as menu items that function similarly to tabs in a tabPanel.

Fig.33. sidebar

Sidebar menu items and tabs

Links in the sidebar can be utilized similarly to Shiny's tabPanels. In other words, when you click a link, the dashboard's body will change to show new content. Here is an illustration of a basic tabPanel:

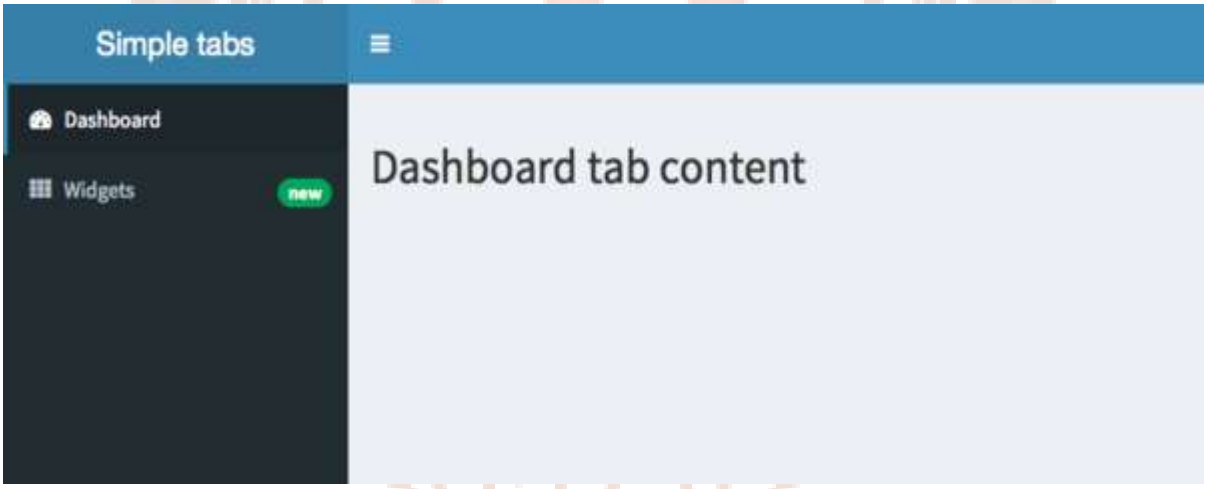

Fig.34. Sidebar menu tabs.

The main body's content changes when the user selects one of the menu items:

Fig.35. Sidebar menu items.

The **sidebarMenu()** function is used to insert the menu items. Make sure that the **tabName** values for a **menuItem** and a **tabItem** match in order to connect them.

```
## ui.R ##
sidebar <- dashboardSidebar(
  sidebarMenu(
    menuItem("Dashboard", tabName = "dashboard", icon = icon("dashboard")),
    menuItem("Widgets", icon = icon("th"), tabName = "widgets",
             badgeLabel = "new", badgeColor = "green")
  )
)

body <- dashboardBody(
  tabItems(
    tabItem(tabName = "dashboard",
      h2("Dashboard tab content")
    ),

    tabItem(tabName = "widgets",
      h2("Widgets tab content")
    )
  )
)

# Put them together into a dashboardPage
dashboardPage(
  dashboardHeader(title = "Simple tabs"),
  sidebar,
  body
)
```

Fig.36. Implementation of sidebar.

If you specify a value for href, a menuItem has additional capabilities beyond controlling tabs. It also has the ability to link to other materials. These external links typically open in a new tab or window in the browser; the new tab option allows you to change this behavior.

```
  menuItem("Source code", icon = icon("file-code-o"),
           href = "https://github.com/rstudio/shinydashboard/")
```

Fig.37. Program for menuItem.

Bookmarking and restoring selected tabs

Shiny now offers the ability to bookmark and restore an application's state as of version 0.14. In a shinydashboard-built project, you must call **sidebarMenu()** with an id in order to bookmark and restore the currently selected **tabItem**. For instance:

```
sidebarMenu(id = "sidebar",
  ....
)
```

Fig.38. Bookmarking and restoring selected tabs.

Dynamic content

RenderMenu and **sidebarMenuOutput** enable the dynamic generation of a sidebar menu. Here is an illustration of a server-generated sidebar application.

```
ui <- dashboardPage(
  dashboardHeader(title = "Dynamic sidebar"),
  dashboardSidebar(
    sidebarMenuOutput("menu")
  ),
  dashboardBody()
)

server <- function(input, output) {
  output$menu <- renderMenu({
    sidebarMenu(
      menuItem("Menu item", icon = icon("calendar"))
    )
  })
}

shinyApp(ui, server)
```

Fig.39. Dynamic content

Inputs in the sidebar

A sidebar can also consist of ordinary inputs, like sliderInputs and textInputs.

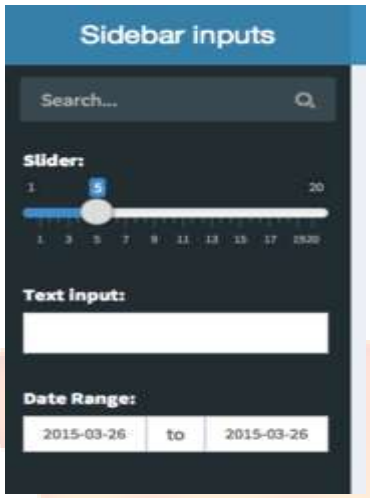

Fig.40. Inputs in the sidebar.

A **sidebarSearchForm**, which is visible at the top in the image above, is another unique sort of input available in shinydashboard. This is essentially a text input that has been particularly formatted, together with an actionButton that looks like a magnifying glass (the icon can be changed with the icon argument).

```
sidebarSearchForm(textId = "searchText", buttonId = "searchButton",
                  label = "Search...")
```

Fig.41. Implementation of sidebarSearchForm.

Disabling the sidebar

If the user don't want a sidebar, user can disable it with:

```
dashboardSidebar(disable = TRUE)
```

Fig.42. Disabling the sidebar.

- **controlbar()**

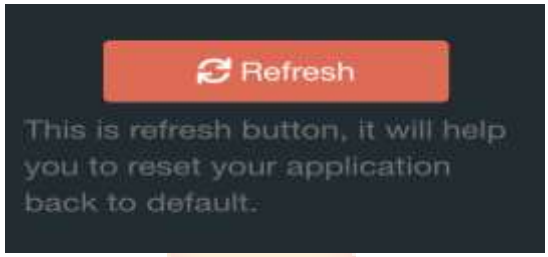

Fig.43. Example of controlbar().

dashboardControlbar build a right sidebar container.

```
dashboardControlbar(
    tagList(
      fluidRow(column(width = 8, offset = 2,br(),
      actionButton("refresh","Refresh",icon("refresh", lib = "font-awesome"),
      width = "100%",
      style = "color: #fff; background-color: #DD6855; border-color: #DD6855"))),
      helpText("This is refresh button, it will help you to reset your application
               back to default.",
      style = "padding-left : 10px; padding-right : 10px;")
      )
  )
```

Fig.44. Implementation of controlbar().

- **The Output Interface**

  body()

```
dashboardBody(
    shinyjs::useShinyjs(),
    shinyjs::extendShinyjs(text = jsRefreshCode, functions = "refresh"),
    shinyalert::useShinyalert(),
    br(),
    fluidRow( uiOutput("boxes")),br(),
    fluidRow(column(width = 6,uiOutput("plot")),
             column(width = 6,uiOutput("plot_area"))),
    fluidRow(column(width = 10, offset = 1,uiOutput("plot_line")))

  )
```

Fig.45. Implementation of Output interface.

The below figure is the example of Stock Market Forecasting Application - Interface UI

Fig.46. Stock Market Forecasting Application - Interface UI.

These templates for stock symbols include the stock, most recent trade price, price change and percentage price change, return date, and volume.

Fig.47. The Output Templates.

The charts that follow are those for individual stock symbols. By using the Study button, the user can select another sticker to explore. We will learn how to use the gradient colour for this. For those plots, I used ggplot2 to make a static plot and Plotly to build an interactive plot.

Fig.48. The Individual Output.

The final graph is a line graph that contrasts every stock symbol.

Fig.49. Stock Symbols Comparison.

- **The Footer**

  footer()

Fig.50. The Application Footer.

Let's look at yet another approach to adding a footer to this programme.

```
dashboardFooter(left = HTML("<span style='font-size:12px;'>
  <a href='https://www.loankimrobinson.com'>Author: Loan Kim Robinson </a></span><br>
  <span style='font-size:12px;'>Email: loankimrobinson@gmail.com</span><br>"),
                          right = "San Diego, Feb 7th 2019")
```

Fig.51. Implementation of footre().

Self-Assessment Questions – 5

9. The _____ function creates the dropdown menus.
10. _____ and _____ enable the dynamic generation of a sidebar menu.

5. SUMMARY

In summary, business analytics and statistics are greatly aided by dashboards. Through a wide range of sources, sizes, and types of data, dashboards offer a window into the understanding and tracking of business indicators. Dashboards make it easier for users to collaborate and make decisions. Data scientists don't need to spend much time presenting the findings because the data story is easily understood by the general public. Additionally, Shiny dashboards' availability on web and mobile platforms increases users' accessibility and mobility.

R-based data specialists can easily incorporate Shiny into their development workflow. Shiny dashboards are adaptable, dynamic, and simple to tailor to each customer's unique requirements. This is partially attributable to the web framework's support for web technologies including HTML, CSS, SCSS, JavaScript, and others. Utilizing Shiny makes it possible to employ modularized codes and functions, quickly prototype ideas, and easily manage dashboards using smaller components.

6. GLOSSARY

Dashboard: All of the user data is shown visually on a dashboard. Although it has a wide range of applications, its main purpose is to present information quickly, like KPIs. The information for a dashboard often comes from a linked database and is shown on its own page.

R-Shiny: The open source R package Shiny offers a beautiful and robust web framework for creating online apps. With the aid of Shiny, you can transform your studies into interactive web applications without having to grasp HTML, CSS, or JavaScript.

Shinydashboard: An R tool called Shiny makes it simple to create dynamic web applications directly from R. Because they are good at assisting businesses in drawing conclusions from the data already available, dashboards are widely used.

7. CONCEPT MAP

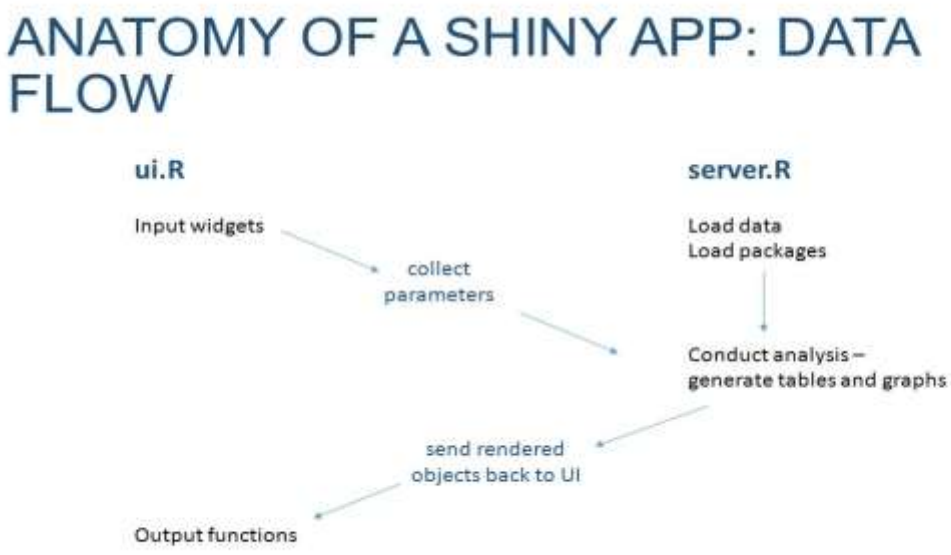

Fig.52: Anatomy of a shiny app.

8. STUDY NOTES & DID YOU KNOW

Did you know?: You may access a full web application framework within the R environment with shiny dashboards. You may quickly turn your R work, analysis, and visualizations, machine learning models, among other things, into web applications that benefit companies.

9.CASE STUDY

Shiny dashboards in healthcare:

The following Shiny app was created by Christian Luz "in the setting of a 1339-bed academic tertiary referral hospital to process the data of over 180,000 admissions." Users of the software can filter patients using one of 17 distinct criteria. Antimicrobial resistance, microbiological tests, and their application can all be researched by users. The investigation's findings can be quickly classified and stratified "to compare predefined patient groups based on specific patient attributes."

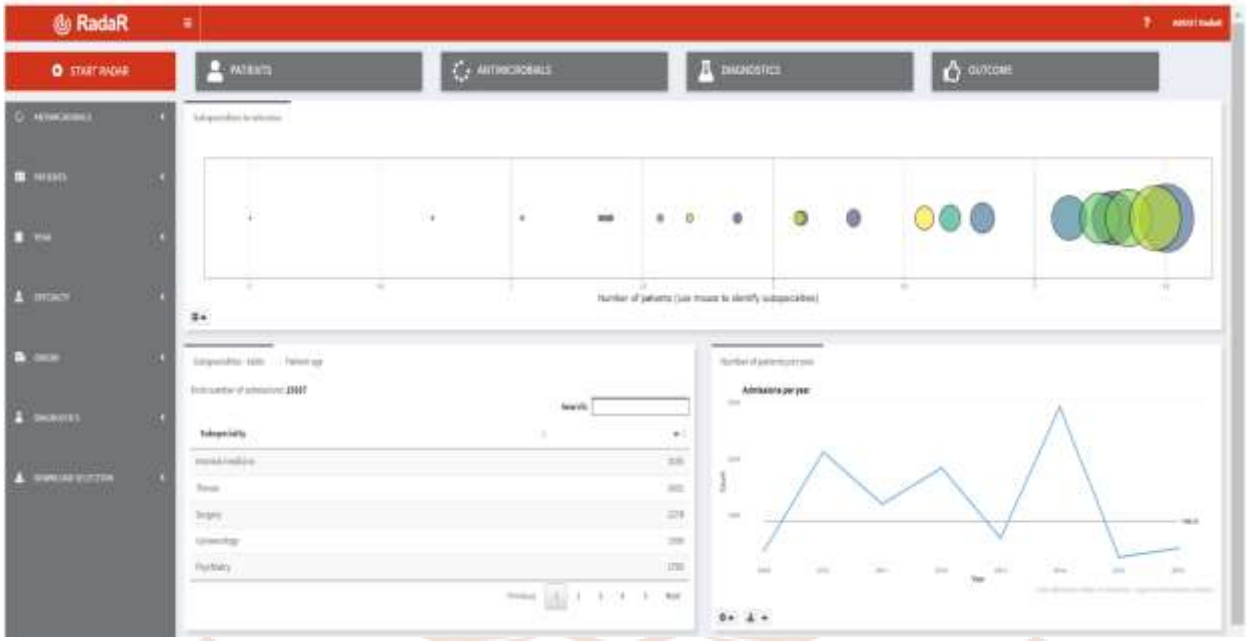

Fig.53: Dashboard for healthcare.

Voter profile:

The Voters Profiles dashboard is a classic illustration of R Shiny in elections. It offers access to graphs, maps, and a novel way to look into the voting profiles in the 2014 elections in Brazil. It received honourable mention status in the 2019 Shiny competition.

Government representatives and others can readily assess the Brazilian elections of 2014 thanks to this dashboard. The results for both the first and second rounds are available by state and city. The number of votes cast for each contender is also displayed as a bar graph.

Votes for governors and senators are displayed on the second tab, State level, while votes for the president are displayed on the first tab, Federal level.

Fig.54: Dashboard for Voter Profile.

10.TERMINAL QUESTIONS

Short Answer Type:

1. List out the 7 basic steps that one should consider when designing an dashboard.
2. Write a basic syntax for creating an dashboard.
3. Explain the skin component of dashboardPage() function.
4. Discuss the footer of dashboardPage() function.

Long Answer Type:

1. Write a program for creating an tabBox in an dashboard
2. Write a program for the dynamic content , notification and task menus.

11. SELF-ASSESSMENT ANSWERS

1. R
2. Dashboards.
3. Dashboards.
4. user interface object and the server function
5. body, a sidebar, and a header.
6. shinyApp()
7. box()
8. tabBox,infoBox,ValueBox
9. dropdownMenu()
10. RenderMenu and sidebarMenuOutput

12. TERMINAL QUESTIONS ANSWERS

1. The 7 basic steps are:

- Begin with the cause
- Use paper and a pen.
- Use prepared solutions.
- Get motivated
- Prioritise data storytelling
- Test first, then repeat the process.
- Look past UI

2.

```
## ui.R ##
library(shinydashboard)

dashboardPage(
  dashboardHeader(),
  dashboardSidebar(),
  dashboardBody()
)
```

3.

skin()

The colour theme is the skin. The backdrop of the sidebar will be light if the skin is light. Depending on the type of app you build, it is simple to select the appearance you like. The plot colour should complement the skin you select for your application. The list of skin tones is provided below.

```
skin = c("blue", "blue-light", "black", "black-light", "purple", "purple-light", "green", "green-light", "red", "red-light", "yellow", "yellow-light", "midnight")
```

4.

footer()

Let's look at yet another approach to adding a footer to this programme.

```
dashboardFooter(left = HTML("<span style='font-size:12px;'>
  <a href='https://www.loankimrobinson.com'>Author: Loan Kim Robinson </a></span><br>
  <span style='font-size:12px;'>Email: loankimrobinson@gmail.com</span><br>"),
                          right = "San Diego, Feb 7th 2019")
```

Long Answer Type:

1. The primary components of dashboard pages are boxes. With the **box()** function, a simple box may be built, and (most) any Shiny UI element can be used as its content.

   tabBox:

   Use a **tabBox** if you need a container to just have tabs for showing various content sets.

```
body <- dashboardBody(
  fluidRow(
    tabBox(
      title = "First tabBox",
      # The id lets us use input$tabset1 on the server to find the current tab
      id = "tabset1", height = "250px",
      tabPanel("Tab1", "First tab content"),
      tabPanel("Tab2", "Tab content 2")
    ),
    tabBox(
      side = "right", height = "250px",
      selected = "Tab3",
      tabPanel("Tab1", "Tab content 1"),
      tabPanel("Tab2", "Tab content 2"),
      tabPanel("Tab3", "Note that when side=right, the tab order is reversed.")
    )
  ),
  fluidRow(
    tabBox(
      # Title can include an icon
      title = tagList(shiny::icon("gear"), "tabBox status"),
      tabPanel("Tab1",
        "Currently selected tab from first box:",
        verbatimTextOutput("tabset1Selected")
      ),
      tabPanel("Tab2", "Tab content 2")
    )
  )
)
```

```
shinyApp(
  ui = dashboardPage(
    dashboardHeader(title = "tabBoxes"),
    dashboardSidebar(),
    body
  ),
  server = function(input, output) {
    # The currently selected tab from the first box
    output$tabset1Selected <- renderText({
      input$tabset1
    })
  }
)
```

2.

Dynamic content

```
dashboardHeader(dropdownMenuOutput("messageMenu"))
```

```
output$messageMenu <- renderMenu({
  # Code to generate each of the messageItems here, in a list. This assumes
  # that messageData is a data frame with two columns, 'from' and 'message'.
  msgs <- apply(messageData, 1, function(row) {
    messageItem(from = row[["from"]], message = row[["message"]])
  })

  # This is equivalent to calling:
  #   dropdownMenu(type="messages", msgs[[1]], msgs[[2]], ...)
  dropdownMenu(type = "messages", .list = msgs)
})
```

Notification menus

A notification contains a **notificationItem** that contains a text notification.the user can also control the status color and icon.

```
dropdownMenu(type = "notifications",
  notificationItem(
    text = "5 new users today",
    icon("users")
  ),
  notificationItem(
    text = "12 items delivered",
    icon("truck"),
    status = "success"
  ),
  notificationItem(
    text = "Server load at 86%",
    icon = icon("exclamation-triangle"),
    status = "warning"
  )
)
```

Task menus

```
dropdownMenu(type = "tasks", badgeStatus = "success",
  taskItem(value = 90, color = "green",
    "Documentation"
  ),
  taskItem(value = 17, color = "aqua",
    "Project X"
  ),
  taskItem(value = 75, color = "yellow",
    "Server deployment"
  ),
  taskItem(value = 80, color = "red",
    "Overall project"
  )
)
```

13. REFERENCES

- https://appsilon.com/dashboards-in-rshiny/
- https://appsilon.com/dashboards-in-rshiny/#using
- https://rstudio.github.io/shinydashboard/structure.html
- https://bookdown.org/loankimrobinson/rshinybook/stock-front-footer.html